



大模型的数学基础

深入浅出的讲解

作为自然语言处理"2-数学基础"课程内容的补充

 面向大学三年级上学期学生

2025-09-28

天工 AI 生成

课程概述

课程内容结构



线性代数

核心线性代数概念、矩阵乘法、内积、范数



优化理论

梯度下降、反向传播、高级优化器



Transformer核心数学

自注意力、多头注意力、位置编码、残差与归一化



概率论与信息论

概率论基础、信息论基础



应用与实践

大模型训练技巧、实际应用案例

学习目标

- 掌握大模型背后的线性代数基础
- 理解优化理论在模型训练中的关键作用
- 深入剖析Transformer架构的数学原理
- 建立概率论与信息论与大模型的联系
- 通过实例应用巩固数学概念的理解

与原课程的关联

本课程作为"2-数学基础（略讲）"的补充，旨在：

- 提供更深入的数学解释
- 聚焦大模型应用中的关键数学概念
- 通过实例和图解增强理解
- 为后续深入学习和应用打下坚实基础

天工 AI 生成

线性代数：大模型的基石

线性代数是描述高维空间中数据结构和变换的数学语言，对于大模型中海量的参数和复杂的计算至关重要。它为我们理解词嵌入、神经网络层操作以及注意力机制等核心组件提供了基础。

核心概念

概念	定义	表示方法	大模型中的应用
标量	单个数值	$x \in R$	学习率、损失函数值、单个神经元的偏置项
向量	有序数组，表示空间中 $x = [x_1, x_2, ..., x_n]^T \in R^n$ 的一个点或方向		词嵌入、单个样本的特征表示
矩阵	二维数组，由行和列组成 $A \in R^{m \times n}$		神经网络层的权重、词向量查找表
张量	多维数组，是标量、向量、矩阵的推广 $A \in R^{d_1 \times d_2 \times ... \times d_k}$		批量处理的图像数据、多头注意力机制中的矩阵集合

大模型中的关键应用

- 词嵌入**
将离散的词语转换为连续的向量表示，捕捉语义和语法信息
- 神经网络层**
矩阵乘法实现层间数据传递，权重矩阵决定特征提取方式
- 注意力机制**
通过内积计算查询与键之间的相似度，实现自注意力层

线性代数与大模型的关系



核心线性代数概念

标量 (Scalar)

定义：单个数值

表示方法： $x \in \mathbb{R}$

大模型应用：学习率、损失函数值、单个神经元的偏置项

向量 (Vector)

定义：有序数组，表示空间中的一个点或方向

表示方法： $x = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}^n$

大模型应用：词嵌入 (Word Embedding)、单个样本的特征表示

矩阵 (Matrix)

定义：二维数组，由行和列组成

表示方法： $A \in \mathbb{R}^{m \times n}$

大模型应用：神经网络层的权重 (连接强度)、词向量查找表

张量 (Tensor)

定义：多维数组，是标量、向量、矩阵的推广

表示方法： $A \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_k}$

大模型应用：批量处理的图像数据、多头注意力机制中的Q/K/V矩阵集合

💡 线性代数是模型的数学语言，掌握这些基本概念是理解模型的基础

天工 AI 生成

矩阵乘法

计算原理

矩阵乘法是神经网络中最核心的运算之一，它描述了信息如何在不同层之间进行线性变换。

一个神经网络层从输入到输出的计算，本质上就是输入向量与权重矩阵的乘法，再加上偏置项。

数学公式

对于矩阵 $A \in \mathbb{R}$ 和 $B \in \mathbb{R}$ ，它们的乘积 $C=AB$ 是一个 $m \times n$ 的矩阵，其中元素 C 定义为：

$$C = \sum_{k=1}^p A_{i,k} B_{k,j}$$

在神经网络中的应用

线性变换

神经网络中的全连接层通过矩阵乘法将输入特征映射到新的特征空间。例如，词嵌入向量 x 经过权重矩阵 W 的变换，得到新的表示 $y = xW$ 。

特征提取

卷积神经网络（CNN）中的卷积操作可以通过特殊的矩阵乘法（Toeplitz矩阵）来实现。

注意力机制

自注意力机制中的 QK 计算就是矩阵乘法，用于衡量查询和键之间的相似度。

矩阵乘法可视化

A (2×3)		B (3×2)		C (2×2)														
<table border="1"><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr></table>	1	2	3	4	5	6	×	<table border="1"><tr><td>7</td><td>8</td></tr><tr><td>9</td><td>10</td></tr></table>	7	8	9	10	=	<table border="1"><tr><td>58</td><td>64</td></tr><tr><td>139</td><td>154</td></tr></table>	58	64	139	154
1	2	3																
4	5	6																
7	8																	
9	10																	
58	64																	
139	154																	
$C[0,1] = A[0,0] \times B[0,1] + A[0,1] \times B[1,1]$																		

内积(点积)

数学定义

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$$

其中 θ 是两个向量之间的夹角

几何意义

内积衡量两个向量之间的相似度或投影关系



在大模型中的应用

相似度计算

在注意力机制中，查询向量 Q 和键向量 K 的内积用于计算它们之间的相似度，从而决定注意力权重：

$$\text{Attention Weights} = \text{softmax}(QK^T / \sqrt{d_k})$$

特征匹配

在推荐系统或信息检索中，内积可以用来衡量用户偏好向量和物品特征向量的匹配程度。向量间夹角越小，内积越大，表示匹配度越高。

神经元激活

传统神经网络中，输入向量与权重向量的加权和（即内积）是神经元激活前的计算：

$$z = \mathbf{x}^T \mathbf{W} + b$$

其中 z 是神经元的输入， x 是输入向量， W 是权重向量， b 是偏置项

王丁 AI 生成

范数

范数是衡量向量或矩阵"大小"或"长度"的数学工具，在机器学习中常用于正则化，以防止模型过拟合。



L1 范数 (曼哈顿范数)

$$||x|| = \sum |x|$$



L2 范数 (欧几里得范数)

$$||x|| = \sqrt{\sum x^2}$$



Frobenius 范数 (矩阵)

$$||A|| = \sqrt{\sum \sum |A|}$$



大模型中的应用

正则化

L1和L2范数正则化（如Lasso和Ridge回归）通过惩罚模型参数的大小来防止过拟合，促使模型学习更稀疏或更平滑的权重。

梯度裁剪

在训练过程中，为了防止梯度爆炸，常常会限制梯度的范数在一个阈值之内。

距离度量

范数可以用来衡量两个向量之间的距离，例如在聚类算法中。

天工 AI 生成

优化理论：学习的引擎

🔧 优化理论的定义

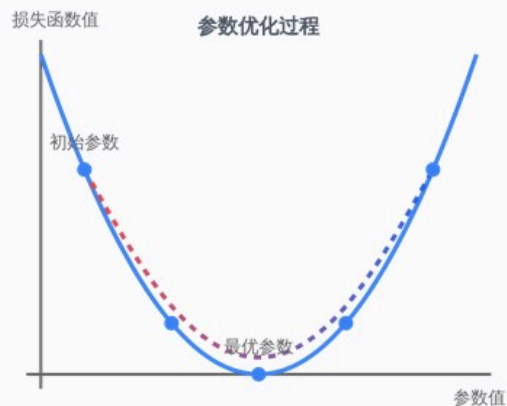
优化理论提供了一套数学工具和算法，指导模型如何调整其内部参数，以最小化预测误差或最大化性能指标。

它如同模型的“学习引擎”，使其能够从海量数据中提取有用的模式。

🌱 在深度学习中的重要性

- 深度学习的核心问题：从数据中学习并做出准确预测
- 优化理论提供了模型参数调整的数学框架
- 决定了模型的训练效率、准确性和泛化能力
- 连接了损失函数设计与模型性能

优化过程可视化



🔍 如何工作

- 定义损失函数（预测误差）
- 计算梯度（损失函数对参数的导数）
- 参数更新（沿着梯度反方向）
- 迭代优化（重复上述步骤）

✂️ 优化目标

- 最小化预测误差
- 最大化性能指标
- 平衡训练误差与泛化能力

天工 AI 生成

梯度下降基本原理

优化目标

梯度下降法是机器学习和深度学习中最基础也是最重要的优化算法之一。其目标是找到使损失函数 $J(\theta)$ 最小的参数 θ 。

基本原理

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

- θ : 模型参数
- η : 学习率 (控制步长)
- $\nabla J(\theta)$: 梯度 (损失函数增长最快的方向)

学习率的重要性

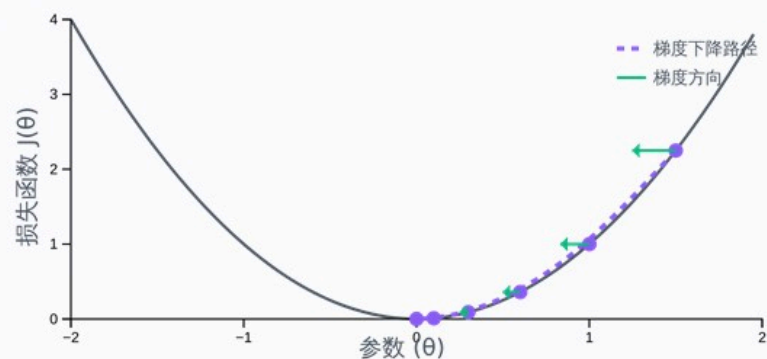
学习率过小

参数更新缓慢, 收敛速度非常慢

学习率过大

可能在最小值附近震荡, 甚至发散

下山的比喻



💡 梯度下降就像在山地上寻找最低点的过程:

- 我们身处一个崎岖的山谷 (损失函数曲面)
- 在每一步, 我们找到最陡峭的下坡方向 (梯度的反方向)
- 沿着这个方向迈出一步 (步长由学习率决定)
- 重复这个过程, 直到找到山谷的最低点 (损失函数的最小值)

梯度下降的变体

梯度下降算法根据使用的训练数据量不同，可分为三种主要变体，它们在收敛速度、稳定性及适用场景上各有优劣。



批量梯度下降 (BGD)

特点

使用所有训练样本来计算梯度

优点

- 每次更新都沿着全局最优方向
- 收敛路径平滑
- 最终能收敛到全局最小值(凸函数)

缺点

- 当训练集非常大时，计算成本高昂
- 训练速度慢
- 不适合在线学习

适用于：小数据集、凸优化问题



随机梯度下降 (SGD)

特点

每次参数更新时，只使用一个训练样本来计算梯度

优点

- 计算速度快，每次更新的开销小
- 能够跳出局部最优解
- 适合在线学习

缺点

- 梯度估计带有噪声
- 收敛路径波动大
- 可能在最小值附近震荡
- 难以精确收敛

适用于：大数据集、实时学习



小批量梯度下降 (MBGD)

特点

使用一批(mini-batch)训练样本来计算梯度

优点

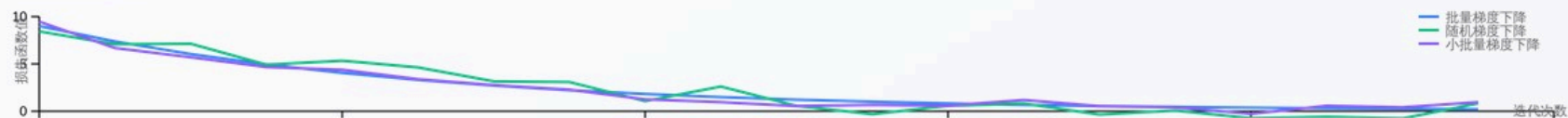
- 结合了BGD和SGD的优点
- 减少计算开销
- 保持相对稳定的收敛方向
- 通过调整批量大小，可在收敛速度和稳定性之间取得平衡

缺点

- 批量大小的选择会影响训练效果

适用于：实践中最常用的方法

收敛路径对比



天工 AI 生成

反向传播算法

什么是反向传播？

反向传播 (Backpropagation, BP) 算法是训练神经网络的核心，解决了如何高效计算复杂网络中所有参数梯度的难题。其核心思想是利用微积分中的链式法则，从网络的输出层开始，逐层向前计算损失函数对每个参数的梯度。

反向传播的三个主要步骤

1 前向传播 (Forward Pass)

输入数据经过网络层层计算，得到最终输出

2 计算损失 (Compute Loss)

根据模型输出和真实标签，计算损失函数值

3 反向传播 (Backward Pass)

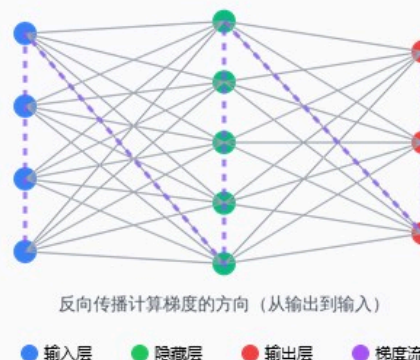
从输出层开始，利用链式法则逐层计算梯度，更新参数

链式法则的应用

链式法则允许我们将一个复杂函数的导数分解为多个简单函数的导数之积。在神经网络中，这使得我们能够高效地计算损失对每个参数的梯度：

$$\partial J / \partial z = (\partial J / \partial z) \times (\partial z / \partial a) \times (\partial a / \partial z)$$

反向传播过程可视化



反向传播的优势

- ✓ 避免了重复计算，通过存储中间计算结果高效完成所有梯度计算
- ✓ 使得训练包含数百万甚至数十亿参数的深度神经网络成为可能
- ✓ 为梯度下降等优化算法提供了高效的梯度计算方法

高级优化器

💡 为了解决朴素梯度下降 (SGD) 收敛慢、易陷入局部最优、对学习率敏感等问题，研究人员开发了许多高级优化器。这些优化器通过引入动量、自适应学习率等机制，显著提高了模型的训练效率和性能。

优化器名称	核心原理	主要优点	潜在缺点
Momentum	引入“动量”项，模拟物理惯性，使参数更新方向在梯度一致时加速，在梯度方向变化时减速。	- 加速收敛 - 减少震荡 - 有助于跳出局部最优	仍需手动调整学习率和动量参数
AdaGrad	为每个参数维护一个学习率，根据参数的历史梯度平方和自适应地调整学习率。梯度大的参数学习率减小快，梯度小的参数学习率减小慢。	- 无需手动调整学习率 - 对稀疏梯度有效	学习率会持续单调递减，可能导致后期学习停滞

💡 其他高级优化器如 RMSProp、Adam 等在实际应用中也取得了良好效果，它们结合了多种优化策略，进一步提高了训练效率和模型性能。

天工 AI 生成

Transformer核心数学

Transformer模型是当前大模型领域的核心基石，其卓越性能很大程度上归功于其独特的架构设计。本章将深入探讨Transformer中涉及的关键数学原理，帮助大家理解这些创新如何共同驱动了模型的强大能力。

架构特点



自注意力机制

允许模型在处理序列的每个元素时，动态地关注输入序列中的其他相关部分



多头注意力

并行运行多个自注意力机制，每个头学习不同的注意力模式



位置编码

为序列中每个词元添加位置信息，弥补了模型缺乏递归结构的缺陷



残差连接与归一化

稳定训练过程，允许训练更深层的网络

数学基础与架构概览



线性代数：理解向量、矩阵、张量在序列处理中的作用



优化理论：梯度下降、注意力机制中的梯度计算与更新



概率论：序列建模与注意力权重计算



信息论：优化目标函数与模型训练

自注意力机制

💡 核心思想

自注意力机制允许模型在处理序列的每个元素时，动态地关注输入序列中的其他相关部分。通过计算查询（Query）与所有键（Key）的相似度，获得注意力分数，再对值（Value）进行加权求和。

√ 计算公式

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d}) \cdot V$$

- **Q(Query):** 查询矩阵，由输入与W相乘得到
- **K(Key):** 键矩阵，由输入与W相乘得到
- **V(Value):** 值矩阵，由输入与W相乘得到
- **dk:** 键向量维度，用于缩放点积

✅ 关键优势

- 捕捉全局依赖关系：不受固定窗口限制
- 动态权重分配：每个词元分配不同注意力
- 并行计算：同时处理所有词元间关系
- 表达能力强：可学习复杂语义关联

📊 矩阵运算过程

1. 输入表示：序列X包含n个词元，维度为d

2. 计算Q、K、V矩阵：

$$Q = XW \in \mathbb{R}$$

$$K = XW \in \mathbb{R}$$

$$V = XW \in \mathbb{R}$$

3. 计算注意力分数：

$$\text{分数} = QK/\sqrt{d} \in \mathbb{R}$$

$$\text{注意力权重} = \text{softmax}(\text{分数}) \in \mathbb{R}^{n \times n}$$

4. 加权求和：输出 = 注意力权重 · V ∈ ℝ

🔍 自注意力可视化



天工 AI 生成

多头注意力机制

√x 数学表达式

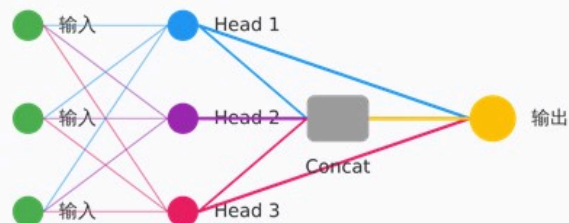
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

参数说明:

- h : 头的数量
- W, W, W : 第 i 个头的查询、键、值权重矩阵
- $\text{Concat}()$: 将所有头的输出在特征维度上拼接
- W : 最终的输出线性变换矩阵

🔗 多头注意力流程



🧩 并行计算原理

多头注意力通过并行运行多个自注意力机制（称为“头”），每个头学习不同的注意力模式，最后将这些头的输出拼接起来并进行线性变换。



★ 从不同“表示子空间”学习的优势

多头注意力可以被理解为模型拥有多个“视角”来观察输入序列。每个头专注于捕捉不同类型或不同粒度的依赖关系。

🧩 多样化表示

不同头可以关注不同的语法结构、语义关联或上下文关系

🔗 增强表达能力

通过整合多个头的输出，模型能够更全面、更丰富地理解输入信息

🔗 捕捉复杂模式

一个头可能关注短距离依赖，另一个关注长距离依赖

🔗 提升模型性能

多头注意力机制通常比单头注意力提供更好的性能

位置编码

🚩 解决方案

Transformer模型完全摒弃了循环和卷积结构，本身不具备处理序列顺序信息的能力。位置编码（Positional Encoding）被引入来弥补这一缺陷。

📊 正弦余弦位置编码公式

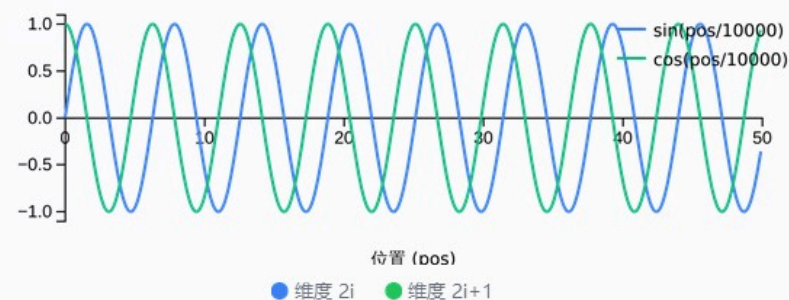
$$PE_{pos, 2i} = \sin(pos / 10000^{2i/d_{model}})$$

$$PE_{pos, 2i+1} = \cos(pos / 10000^{2i/d_{model}})$$

参数说明：

- **pos**: 词元在序列中的位置（从0开始）
- **i**: 位置编码向量中的维度索引
- **dmodel**: 词嵌入的维度

📈 正弦曲线示意图



💡 关键原理与优势

- ➡ 对于任何固定的偏移量 **k**，位置 **pos+k** 的位置编码可以表示为位置 **pos** 的位置编码的线性函数。这使得模型能够轻松地学习到相对位置信息。
- 📉 随着维度 **i** 的增加，频率逐渐降低，使得位置编码能够捕捉不同尺度的位置信息。
- 🔧 这种编码方式是可扩展的，即使遇到比训练时更长的序列，也能生成有效的位位置编码。

残差与归一化



残差连接

由He等人在2015年的论文《Deep Residual Learning for Image Recognition》中首次提出。

核心思想

在网络层之间添加直接连接，允许输入信号绕过一个或多个层直接传播到更深的层。

数学表达

如果一个子层的输入是 x ，其输出是 $\text{Sublayer}(x)$ ，则残差连接的输出为：

$$x + \text{Sublayer}(x)$$



优势

- > 训练更深层的神经网络
- > 稳定训练过程
- > 解决梯度消失/爆炸问题
- > 保留原始输入信息



层归一化

在Transformer模型中广泛采用的另一种技术，与残差连接配合使用。

原理

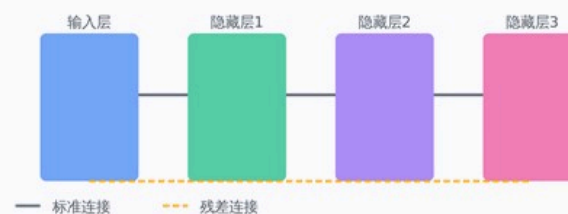
在每个层的输入上应用归一化，稳定训练过程。

与残差连接的配合

在Transformer中，层归一化通常应用于每个子层的输入，而残差连接则将原始输入与子层输出相加。



工作原理图示



概率论基础

概率论是大模型理解的基础，为语言模型和生成模型提供了数学基础。它帮助模型处理不确定性，从数据中学习并生成合理的输出。

核心概念



条件概率

条件概率 $P(A|B)$ 表示在事件B发生的情况下，事件A发生的概率。在大模型中，用于计算给定上下文的下一个词的概率。

$$P(A|B)$$

在B发生的条件下A发生的概率



贝叶斯定理

贝叶斯定理描述了如何根据新证据更新概率： $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ 。在大模型中用于文本生成和语言理解。

$$P(A|B) = P(B|A)P(A)/P(B)$$

后验 = 似然 × 先验 / 证据

应用



语言模型

- 计算句子或文本序列的概率
- 评估语义相关性
- 文本生成 (如GPT系列)
- 机器翻译



生成模型

- 基于条件概率的语言生成
- 变分自编码器 (VAE)
- 生成对抗网络 (GAN)
- 扩散模型

信息论基础

熵 (Entropy)

衡量随机变量的不确定性或信息量

$$H(X) = -\sum p(x) \log p(x)$$

交叉熵 (Cross-Entropy)

衡量两个概率分布之间的差异

$$H(p, q) = -\sum p(x) \log q(x)$$

KL散度 (KL Divergence)

衡量两个概率分布之间的相对熵

$$D(p||q) = \sum p(x) [\log p(x) - \log q(x)]$$

在大模型中的应用

损失函数设计

交叉熵损失函数是分类问题的标准选择，KL散度用于自编码器和变分自编码器

模型评估

使用信息论指标评估模型性能，如困惑度 (perplexity)

注意力机制

信息论概念用于解释自注意力机制如何分配“注意力”

相对熵可视化

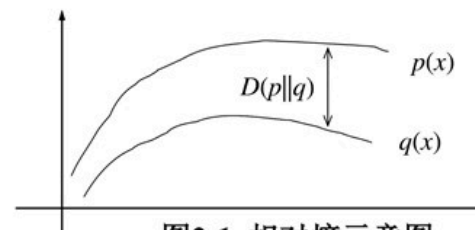


图2-1. 相对熵示意图

相对熵 $D(p||q)$ 表示分布 p 和 q 之间的差异

天工 AI 生成

大模型训练技巧

💡 大模型训练面临着计算资源受限、内存瓶颈和训练时间长等挑战。以下数学技巧能够有效解决这些问题，提高训练效率。



梯度累积

✚ 数学原理：将多个小批量的梯度累加后再更新模型参数

$$\theta = \theta - \eta \sum_{i=1}^n \nabla_{\theta} J(\theta)$$

✓ 优势：相当于使用更大的批量大小，而不增加内存消耗

🔧 实现：累积 n 个小批量的梯度，每 n 步更新一次参数



混合精度训练

✚ 数学原理：使用FP16（半精度）和FP32（单精度）混合表示

前向传播和梯度计算：FP16参数更新和累加：FP32

✓ 优势：减少内存占用，加速矩阵运算，尤其在支持Tensor Core的硬件上

⚠ 数值稳定性：需要使用损失缩放(loss scaling)技术避免梯度下溢



并行计算策略

🔧 数据并行(Data Parallel)：跨设备复制模型，分割数据

每个设备: $\nabla_{\theta} J(\theta)$ 全局:

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} J_i(\theta)$$

🔧 模型并行(Model Parallel)：将模型分割到多个计算设备上

🔧 流水线并行(Pipeline Parallel)：将模型层分组并在不同设备上顺序执行

★ 实施建议

✓ 技巧组合使用效果最佳

✓ 根据硬件资源选择合适策略

✓ 监控训练稳定性和收敛性

总结与展望

核心内容回顾

线性代数基础

向量、矩阵、张量及其运算构建了大模型的骨架，为数据表示和变换提供了数学语言。

优化理论

梯度下降、反向传播和高级优化器构成了模型学习的“引擎”，驱动参数不断优化。

Transformer数学原理

自注意力、多头注意力、位置编码等机制使模型能够捕捉序列中的长距离依赖关系。

未来发展趋势

- 数学将继续在**模型创新**中发挥关键作用
- 更高效的**优化算法**将进一步提升大模型训练效率
- **可解释性研究**将依赖更复杂的数学工具
- 跨学科数学方法将为大模型带来新的突破

实践建议

通过实践加深对数学原理的理解：

编程实现

使用PyTorch或TensorFlow复现简单模型组件

可视化分析

绘制注意力权重热力图，理解模型决策过程

实验探索

调整模型参数，观察对性能的影响

团队合作

与同学协作，共同攻克复杂数学问题

★ 结语

数学是理解大模型内部机制的钥匙，也是推动其不断发展的引擎。通过掌握这些基础数学原理，你已经向成为AI领域专业人才迈出了重要的一步。

“数学是AI的语言，实践是理解的桥梁”